Learning what to change: the poster, explained

REVIEW DRAFT, 18 October; for Toni, Chris and Pierce before anything is printed. Read the plain text aloud. Grey lines in brackets are notes to you. Every number is in `records/CORA_SUMMIT_CLAIM_LEDGER_20261017.md`, with its scope.

If you only have 20 seconds

I study how a reasoning system can tell *what to change* when it fails. Is the problem the way it sees the objects, the rule it applies, or simply not having enough evidence? My system, CORA, solves puzzles by writing small programs. It can now notice when it has only memorized an answer, build a missing rule, check the rule on examples it has not seen, and use it. That worked on one practice puzzle and on one real ARC puzzle. What I have not shown yet is that learning from the failure is what made the difference: a version that ignores the failure did just as well. That gap is my research question.

Part 1. The question

1. The problem

[column 1, top]

People can learn a new kind of reasoning from a handful of examples, even when the meaning of the examples is new. For a machine this breaks into three questions:

1. **Representation.** Can it find the right things to reason about (objects, parts, relations) in data it has not seen before?
2. **Rules.** Can it discover the executable rules that act on those things?
3. **Revision and transfer.** When it fails, can it tell which of these was wrong and fix the right one? And does that skill of fixing carry over to a domain with completely different meaning, for example from visual puzzles to molecular measurements?

These questions were shaped in discussion with Toni Kazic.

2. Why one solved puzzle proves little

Many different programs can reproduce every example and still disagree on a new case. On 60 practice puzzles, CORA's object reasoner often found programs that matched every example perfectly and were wrong on the test. That happened in 26 puzzles (27 without its saved library). It had memorized the examples instead of finding the rule.

A failure can come from three places, and each needs a different fix:

- the **representation**: the objects were cut up wrongly;
- the **rule**: the right operation was missing;
- the **evidence**: the examples do not decide between several rules.

Choosing the wrong fix can break things that used to work.

Part 2. Why ARC is a good place to study this

3. The puzzles

[any grid on the poster]

Each ARC puzzle gives two to five example pairs of coloured grids and one new input. The solver must produce the exact output. There is no partial credit and no training on the answer. That makes ARC a controlled laboratory:

- the evidence is sparse;
- every answer can be checked exactly;
- many puzzles need a transformation that no fixed list of operations covers.

It is not the end goal. It is the place to test the reasoning procedure before trying it in science, where checking is much harder.

Part 3. What CORA actually demonstrates

4. Four achievements, four different statuses

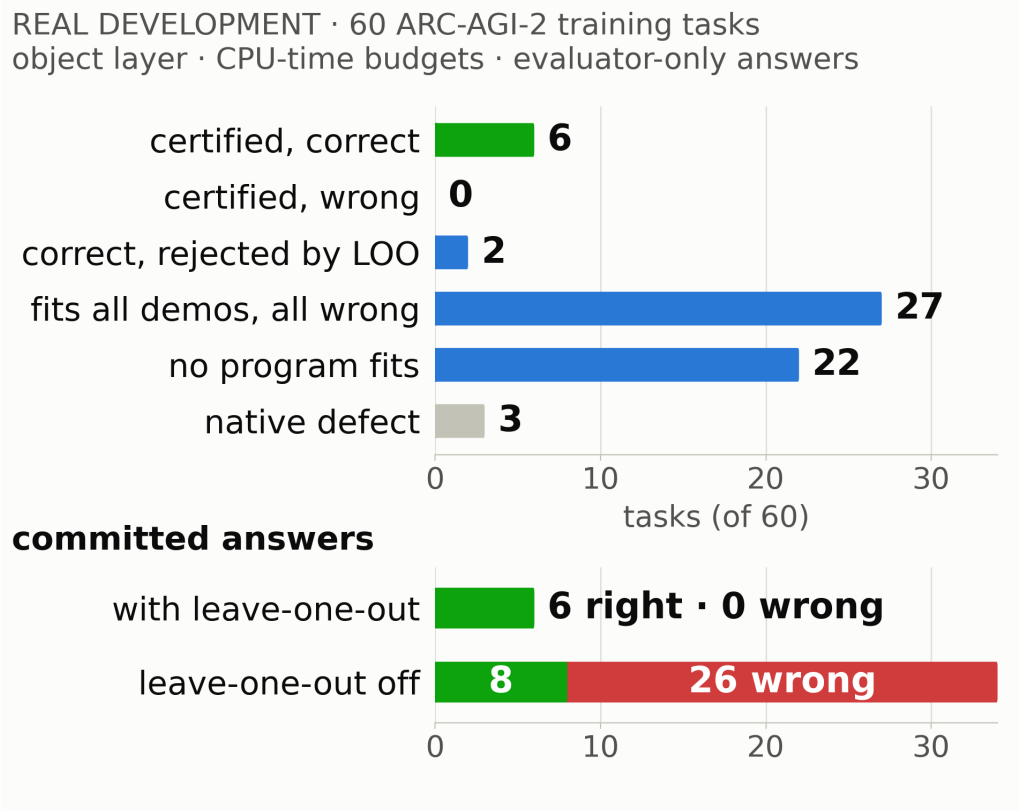
[the table in column 1]

achievement	status today
detect certain reasoning failures	built, with limits
construct and verify an executable correction	one synthetic and one exposed real puzzle; 0 of 58 on an independent set
learn to choose better revisions from failures	**not established**
transfer a revision policy to biology	**not tested**

Keeping these four apart is what makes the larger idea credible.

5. Where CORA fails on real puzzles

[the bar chart in column 2]

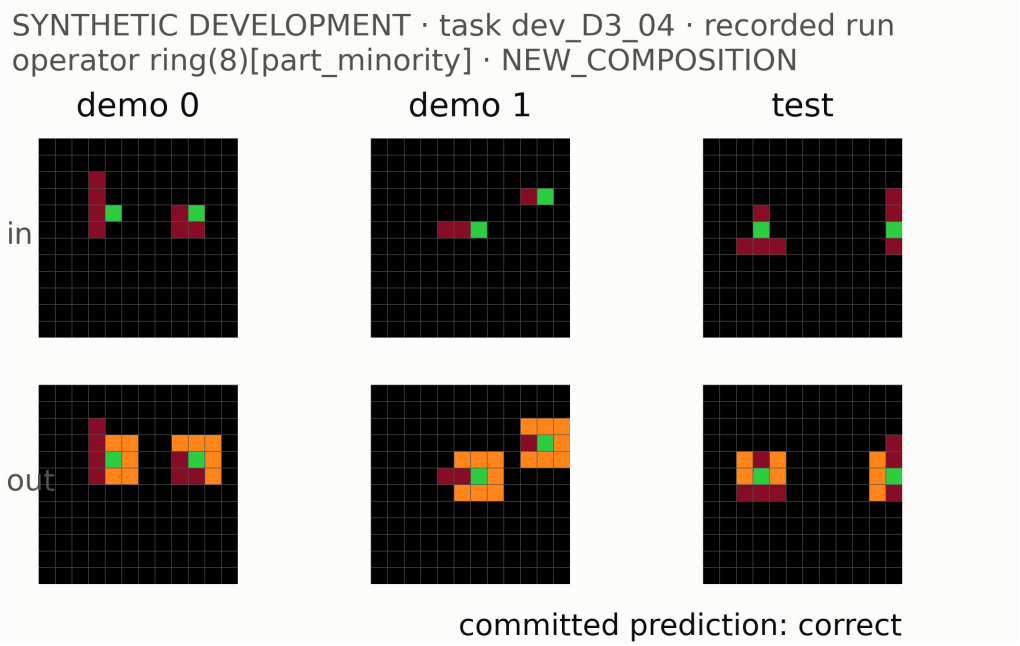


I measured, step by step, where CORA's object reasoner fails on 60 real practice puzzles.

- Its own check is careful: it accepted 6 answers, and all 6 were right.
- The big problem is memorizing. In 26 puzzles (27 without the saved library, as in the chart) every program that fitted the examples was wrong.
- A correct candidate existed on 8 puzzles. With the check switched off it would have answered 8 right and 25 wrong (26 in the chart's configuration).
- Giving it three times the search time fixed none of the memorized puzzles.

6. Building a missing rule: the practice puzzle

[the small figure, if shown]

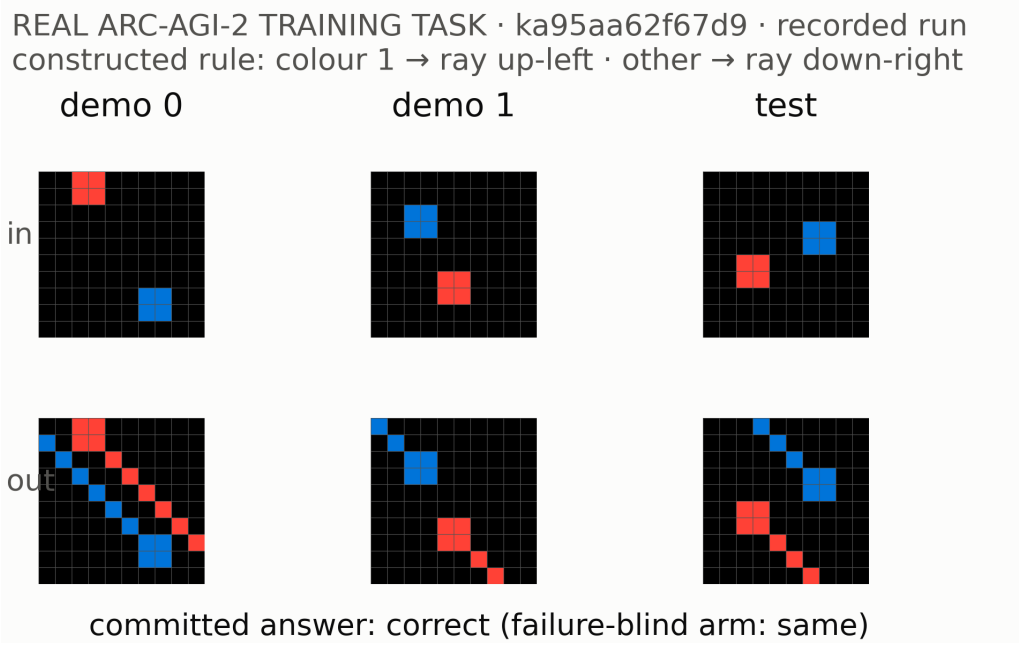


When the solver could only memorize, CORA looked at exactly which objects it had memorized. From a fixed list of building blocks (37,750 possible rules) it built one: a ring around the odd-coloured cell. It plugged that rule into the unchanged solver and tested it three times, each time hiding one example, rebuilding the rule from the others and predicting the hidden one. All three passed, and the final answer was right. Removing the rule broke it again.

The rule is a single step, not a complex one. A version that ignored the failure record found the same rule.

7. A real puzzle solved by a built rule

[the grid figure in column 2; demo: `run_demo.sh -real`]



This is a real ARC training puzzle. CORA's solver fails it, even with extra search time. My failure analysis found that each object's change could be described, but not all of them by one rule. So I let a rule say *where* it applies as well as *what* it does. CORA built: "blue objects (colour 1) shoot a ray to the upper left; the others shoot one to the lower right". It passed all three hidden-example checks, the solver used it, the solver was restored afterwards, and the answer was right.

The limits:

- I found this puzzle through my own analysis;
- it is a training puzzle, so it is not a test of generalization;
- a version that ignores the failure record found the same rule, from the same evidence;
- it was built after my Kaggle entry, so it is not part of that entry.

8. Does learning from the failure actually help?

[the red "not established" block]

This is the central question, and the honest answer today is: not shown.

- On the practice puzzles, a constructor that saw the same information but not the failure record matched every certified answer.
- A pilot on puzzles where the solver understood some objects but not others failed its pre-set criterion.
- In a frozen 10-puzzle test of the new where-and-what rules, learning from the failure got 2 right and the failure-blind version got 1 right. The blind version abstained once, because 15 equally simple rules disagreed. One puzzle of difference is not evidence (a sign test gives $p = 0.5$).
- I wrote a simple rule book that chooses which kind of fix to try from the solver's trace. Tested on recorded runs, it missed the one puzzle that extra search time had fixed: 6 right against 7.

So **building a rule** and **learning which revision to make** are different capabilities. Only the first is demonstrated.

9. Why construction rarely fires

In the Kaggle practice run, the construction step could apply to 62 puzzles:

- 20 left no trace of memorizing;
- 39 left traces that no rule fitted;
- 2 ran out of time;
- 1 rule fell apart in its checks.

Looking closely at 25 real cases, 18 needed operations the building-block list simply does not contain. That is a limit of the language I gave it, not of search. Learning the language itself is the larger problem, which my separate Step B experiment studies.

10. What is designed by hand, and what is automatic

- **Designed by me:** the trigger (“objects the solver could only memorize”), the list of building blocks, and the checks.
- **Automatic:** the choice of rule, the choice of where it applies, and its rebuilding in every check.
- Hiding one of three examples at a time is a consistency check, not a proof that the rule holds everywhere.

Part 4. What others already do

11. Close relatives

[“Where this sits”, column 1]

- **Popper** (Cropper and Morel) learns programs by learning from failed hypotheses.
- **DreamCoder** (Ellis and colleagues) and **LILO** (Grand and colleagues) grow libraries of reusable program pieces.
- **Model Discovery Agent** (Kevin Murphy) uses a model’s failures to revise hypotheses and design experiments. It is the closest idea to the broader question here.
- Large neural reasoners and scientific agents are strong and improving fast. They can use tools and check their own work, and I do not claim otherwise.

What I add is narrower and testable: measuring whether a system chooses *which component* to revise better than an equally equipped system that does not use the diagnosis. Measuring the damage done by wrong revisions, and the cost, is part of the test.

Part 5. The experiment that would answer the larger question

12. The independent test, and its negative result

[column 3, "An independent test"]

On 58 practice puzzles, chosen by a fixed rule before anything ran, I compared:

- the solver alone, and with extra search;
- learning from the failure;
- an equally equipped version that ignores the failure;
- a version given someone else's failure record;
- a new version that learns *what* to change from what the solver could only memorize, and *where* it applies from the contrast with what the solver already explains.

The comparisons were decided in advance, with corrections for testing several things at once.

The result: no version that builds rules got a single puzzle right, 0 of 58. None got one wrong either; the original solver alone got 1. So the real-puzzle success did not generalize, and learning from the failure showed no advantage. Construction rarely even proposed a rule (at most 6 of the 58), and the solver usually rejected what was proposed. The bottleneck is upstream: CORA rarely has the right evidence, or a rich enough language, to build the missing rule. Choosing the revision cannot help until that is fixed.

13. From puzzles to science (proposed, not tested)

[column 3, "Scientific prediction"]

The same question matters in biology. When a model of molecular measurements fails, should it change how it represents the measurements, revise its hypothesis, or ask for a new experiment?

What might transfer is the *revision policy* and the checking discipline, not any puzzle operation. I separate four levels:

1. the same software runs in both domains;
2. the same method trained separately in each;
3. a policy learned on ARC, frozen, improves revision decisions on the biological task;
4. those decisions improve independently validated predictions.

Only the fourth would be a biological result, and none of them has been tested.

A biological check also differs from an ARC check: it needs a tolerance, real replicates, held-out conditions and, in the end, an experiment. The concrete problem is being defined with Pierce and Toni; nothing here uses their data.

Part 6. Benchmark context

- Official Kaggle ARC-AGI-2 public score of my own solver, native CORA: **0.00**.
- A careful reproduction of a published neural method (NVARC, Qwen3-4B): **27.64**. It is not my invention and not CORA.
- On 18 October I submitted a research entry that runs CORA's construction step. It scored **0.00**, as expected: in its practice run it added no answers, and the where-and-what rule above was built after it.

Part 7. How to reproduce what you see

- **Review poster and this explainer:** `docs/summit_20261023_reframe/poster_review.tex` and `explainer_review.tex`, built with pdflatex. The figures are in `docs/summit_poster/figures/`, drawn from hash-pinned records (`scripts/summit_figures.py`; hashes in `figures/PROVENANCE.json`).
- **Both demos** (offline replays, numpy only; each refuses to run if a recorded file's hash differs): `sh docs/summit_demo_20261023/run_demo.sh` (practice puzzle) and `run_demo.sh -real` (the real puzzle).
- **The experiments:** the failure analysis is `records/CORA_REVISION_ROOT_CAUSE_20261018.md`; the frozen 10-puzzle test is `records/CORA_FHR_GUARD_PROBE_PROTOCOL_20261018.md`; the running 58-puzzle test is `records/CORA_FHR_GUARD_GENERALIZATION_PROTOCOL_20261018.md`.